

## Punto 2.4 - Relaciones uno a uno

**Ejercicio guiado:** añadir y configurar dos relaciones uno-a-uno en `AceriaData`: `OrdenFabricacion` -> `DetalleOrden` y `OrdenFabricacion` -> `CertificadoCalidad`. El ejercicio identifica principal y dependiente, hace explícita la clave foránea, comprueba la unicidad de `OrdenId`, aplica eliminación en cascada y carga ambas referencias con `Include`.

**Contexto del proyecto:** 2.4 continúa directamente desde 2.3. La relación uno-a-muchos entre orden y planchas ya está configurada. En este punto se incorporan `DetalleOrden` y `CertificadoCalidad` como dependientes uno-a-uno de `OrdenFabricacion`. Una orden puede existir sin detalle o sin certificado; si alguno de esos dependientes existe, su `OrdenId` es obligatorio y debe identificar una única orden.

### Objetivos de aprendizaje

- Diferenciar una relación uno-a-uno de una relación uno-a-muchos.
- Identificar principal y dependiente en una relación 1:1.
- Entender por qué el dependiente contiene la clave foránea.
- Configurar `HasOne`, `WithOne` y `HasForeignKey<TDependiente>`.
- Comprender la relación entre `WithOne` y el índice único sobre la FK.
- Permitir que la navegación desde la orden sea opcional sin convertir en opcional la FK del dependiente.
- Configurar `DeleteBehavior.Cascade`.
- Añadir `DetalleOrden` y `CertificadoCalidad` sin duplicar la configuración de la misma relación.
- Revisar la migración real y los dos índices únicos generados.
- Insertar y cargar una orden con detalle y certificado.
- Verificar que SQL Server rechaza un segundo certificado para la misma orden.

### Paso 1: Abrir el estado autónomo de 2.4

```
cd M02/PROYECTO/2.4
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El estado 2.4 contiene la solución completa del punto. Si estás construyendo el ejercicio manualmente desde 2.3, aplica sobre tu copia los cambios de los pasos siguientes y genera después la migración incremental.

### Paso 2: Añadir las navegaciones uno-a-uno a `OrdenFabricacion`

La entidad principal incorpora dos referencias opcionales:

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();

    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
}
```

Las dos navegaciones llevan `?` porque una orden puede crearse antes de disponer de su detalle técnico o de su certificado de calidad. Esa opcionalidad pertenece al **principal**: no obliga a que toda orden tenga un dependiente.

No significa que un `DetalleOrden` o un `CertificadoCalidad` existente pueda quedar sin orden. Esa regla se expresará en sus propias configuraciones mediante una FK `int` no anulable y `.IsRequired()`.

### Paso 3: Crear `DetalleOrden` y exponerlo en el `DbContext`

La entidad dependiente del detalle es:

```
public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

y el contexto la expone con:

```
public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
```

Interpreta cada parte:

- `Id` es la clave primaria propia del detalle.
- `OrdenId` es la clave foránea hacia `OrdenFabricacion`.
- `Orden` es la navegación de referencia hacia el principal.
- `ComposicionQuimica` es requerida.
- `Notas` es opcional.

`DetalleOrden` es el dependiente porque contiene `OrdenId`.

### Paso 4: Configurar una sola vez la relación `OrdenFabricacion-DetalleOrden`

La configuración definitiva se hace desde el dependiente:

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);

    entity.Property(d => d.ComposicionQuimica)
        .IsRequired()
        .HasMaxLength(200);

    entity.Property(d => d.Notas)
        .HasMaxLength(500);

    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

Línea a línea:

1. `HasOne(d => d.Orden)` selecciona la referencia del dependiente al principal.
2. `WithOne(o => o.Detalle)` establece que la navegación opuesta también es una única referencia.

3. `HasForeignKey<DetalleOrden>(d => d.OrdenId)` identifica explícitamente a `DetalleOrden` como dependiente y a `OrdenId` como FK.
4. `OnDelete(DeleteBehavior.Cascade)` propaga al detalle la eliminación física de su orden.
5. `IsRequired()` expresa que un detalle existente debe pertenecer a una orden.

No se configura una segunda vez la misma relación desde `OrdenFabricacion` con `IsRequired(false)`. La navegación `DetalleOrden? Detalle` ya expresa que **la orden puede no tener detalle**, mientras que `DetalleOrden.OrdenId` sigue siendo obligatorio cuando el dependiente existe.

## ▮ Paso 5: Crear CertificadoCalidad y exponerlo en el DbContext

La segunda relación uno-a-uno usa otra entidad dependiente:

```
public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

El contexto añade:

```
public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
```

Aquí también `OrdenId` reside en el dependiente. `NumeroCertificado` es un dato de negocio del certificado, pero en 2.4 no se adelantan las claves alternativas del punto 2.8.

## ▮ Paso 6: Configurar la relación OrdenFabricacion-CertificadoCalidad

La configuración real del estado es:

```
modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);

    entity.Property(c => c.NumeroCertificado)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(c => c.OrganismoCertificador)
        .IsRequired()
        .HasMaxLength(100);

    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

La forma es paralela a `DetalleOrden`:

- `OrdenFabricacion` es principal.
- `CertificadoCalidad` es dependiente.
- `OrdenId` es la FK.
- una orden puede existir sin certificado;
- si el certificado existe, debe apuntar a una orden válida.

## Paso 7: Entender qué hace realmente que la relación sea uno-a-uno

Una FK normal permite que varias filas dependientes apunten al mismo principal. Para que la cardinalidad sea 1:1, `OrdenId` debe ser **único** en cada tabla dependiente.

La combinación:

```
.WithOne(...)
.HasForeignKey<DetalleOrden>(d => d.OrdenId)
```

hace que EF Core modele una FK única para `DetallesOrden.OrdenId`. Lo mismo ocurre con `CertificadosCalidad.OrdenId`.

En SQL Server el resultado se materializa mediante índices únicos. Si se intenta insertar un segundo dependiente con la misma `OrdenId`, la base de datos rechaza la operación.

## Paso 8: Generar la migración incremental M2\_2\_4

Si estás reproduciendo el laboratorio desde una copia de 2.3:

```
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations add M2_2_4
```

En el estado entregado la migración ya está generada:

```
Migrations/20260927204735_M2_2_4.cs
```

No vuelvas a ejecutar `migrations add M2_2_4` sobre la solución final del punto. El comando representa el paso incremental que se realiza al construir 2.4 desde 2.3.

## Paso 9: Revisar las tablas, claves foráneas e índices únicos de la migración

`M2_2_4` crea dos tablas nuevas:

- `DetallesOrden`;
- `CertificadosCalidad`.

Para `DetallesOrden`, la migración define la FK:

```
table.ForeignKey(
    name: "FK_DetallesOrden_OrdenesFabricacion_OrdenId",
    column: x => x.OrdenId,
    principalTable: "OrdenesFabricacion",
    principalColumn: "Id",
    onDelete: ReferentialAction.Cascade);
```

y crea:

```
migrationBuilder.CreateIndex(
    name: "IX_DetallesOrden_OrdenId",
    table: "DetallesOrden",
    column: "OrdenId",
    unique: true);
```

Para `CertificadosCalidad` aparece la misma estructura:

```
migrationBuilder.CreateIndex(
    name: "IX_CertificadosCalidad_OrdenId",
    table: "CertificadosCalidad",
    column: "OrdenId",
    unique: true);
```

Los dos `unique: true` son evidencia directa de que una misma orden no puede tener dos detalles ni dos certificados.

## Paso 10: Aplicar la migración y comprobar el esquema en SQL Server

Ejecuta:

```
dotnet ef database update
dotnet ef migrations list
```

Comprueba en SQL Server Object Explorer:

- tabla `dbo.DetallesOrden`;
- tabla `dbo.CertificadosCalidad`;
- FK de cada tabla hacia `OrdenesFabricacion(Id)`;
- `IX_DetallesOrden_OrdenId` con `Unique = True`;
- `IX_CertificadosCalidad_OrdenId` con `Unique = True`.

El esquema sigue gobernado por Migrations. No se usa `EnsureCreated()` para sustituir el historial.

## Paso 11: Insertar una orden con sus dos dependientes

El estado 2.4 crea una única orden y anida ambas referencias:

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Pendiente",

    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },

    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    }
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

Al guardar el grafo, EF Core inserta primero la orden y propaga su clave a `DetalleOrden.OrdenId` y `CertificadoCalidad.OrdenId`.

## Paso 12: Cargar Detalle y Certificado con Include

La comprobación funcional carga las dos referencias:

```
var cargada = context.OrdenesFabricacion
    .Include(o => o.Detalle)
    .Include(o => o.Certificado)
    .Single(o => o.NumeroOrden == "OF-M2-0001");

global::System.Console.WriteLine(
    $"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} | " +
    $"Certificado: {cargada.Certificado?.NumeroCertificado}");
```

La salida debe contener:

```
2.4 OK | Detalle: C: 0.45%, Mn: 0.75% | Certificado: CERT-0001
```

`Include` es necesario porque las referencias no se cargan por arte de magia en una consulta nueva. El ejercicio hace explícito qué parte del grafo necesita.

## Paso 13: Resolver el reto de unicidad con un segundo certificado

**Reto:** intentar guardar un segundo `CertificadoCalidad` con la misma `OrdenId` y comprobar que SQL Server lo rechaza.

La solución del punto contiene el bloque comentado `RETO 2.4 - CERTIFICADO ÚNICO`. Utiliza un ámbito y un `DbContext` nuevos para que la prueba llegue realmente a la restricción única de la base de datos, sin que el relationship fixup del `ChangeTracker` altere el escenario:

```
using var retoScope = provider.CreateScope();
var retoContext =
    retoScope.ServiceProvider.GetRequiredService<AceriaDbContext>();

retoContext.CertificadosCalidad.Add(new CertificadoCalidad
{
    OrdenId = orden.Id,
    NumeroCertificado = "CERT-DUPLICADO",
    FechaEmision = DateTime.Today,
    OrganismoCertificador = "Laboratorio duplicado"
});

try
{
    retoContext.SaveChanges();
    global::System.Console.WriteLine(
        "Reto 2.4 ERROR: se permitió un segundo certificado.");
}
catch (DbUpdateException)
{
    global::System.Console.WriteLine(
        "Reto 2.4: segundo certificado rechazado");
}
```

Resultado esperado:

```
Reto 2.4: segundo certificado rechazado
```

La prueba confirma en el proveedor real que el índice único sobre `CertificadosCalidad.OrdenId` protege la cardinalidad 1:1.

## Errores comunes

| Error                                                                           | Causa                                                                               | Corrección                                                                            |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Se configura la misma relación dos veces con reglas distintas                   | Se intenta expresar la opcionalidad desde ambos extremos                            | Configurar una sola vez desde el dependiente                                          |
| Se usa <code>IsRequired(false)</code> sobre una FK <code>int</code> no anulable | Se confunde “la orden puede no tener detalle” con “un detalle puede no tener orden” | Mantener <code>OrdenId</code> obligatorio y la navegación del principal como nullable |
| Aparecen dos detalles para una orden                                            | La FK no tiene restricción única                                                    | Comprobar <code>IX_DetallesOrden_OrdenId</code> con <code>Unique = True</code>        |

| Error                                                                          | Causa                                 | Corrección                                                                                    |
|--------------------------------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------|
| El certificado duplicado se sustituye en memoria en vez de llegar a SQL Server | Se reutiliza el mismo grafo rastreado | Ejecutar la prueba de unicidad con un <code>DbContext</code> separado                         |
| Detalle o certificado aparecen <code>null</code> al consultar                  | Se omitió <code>Include</code>        | Incluir explícitamente las navegaciones necesarias                                            |
| Se añade <code>OrdenAleacion</code>                                            | Se adelanta el punto 2.5              | Mantener 2.4 exclusivamente en relaciones 1:1                                                 |
| Se usa <code>EnsureCreated()</code>                                            | Se evita el historial incremental     | Usar <code>Migrations</code> y <code>Database.Migrate()</code> / <code>database update</code> |

## Analogía final

Una relación uno-a-uno se parece al expediente técnico único de una orden de fabricación. La orden puede existir mientras ese expediente todavía se está preparando, pero, una vez creado, el detalle pertenece a una sola orden y no puede duplicarse para la misma `OrdenId`. El certificado de calidad sigue la misma regla: una orden puede estar pendiente de certificación, pero el certificado que exista queda ligado de forma única a ella. Los índices únicos son la regla física que impide que aparezcan dos expedientes o dos certificados para la misma orden.

## Resultado esperado

Al finalizar 2.4, `AceriaData` conserva todo lo construido en 2.3 y añade `DetalleOrden` y `CertificadoCalidad`. La migración `M2_2_4` crea las dos tablas, sus claves foráneas con `Cascade` y los índices únicos sobre `OrdenId`. La aplicación carga ambas referencias mediante `Include` y termina con la evidencia `2.4 OK`. El reto confirma que SQL Server rechaza un segundo certificado asociado a la misma orden.

## Conexión con el siguiente punto

Con las relaciones 1:N y 1:1 ya definidas, 2.5 añadirá la relación muchos-a-muchos entre `OrdenFabricacion` y `Aleacion` mediante la entidad intermedia explícita `OrdenAleacion`, donde la propia relación tendrá datos de negocio.

## Código acumulativo completo del estado 2.4

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

```

```

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
            entity.Property(e => e.Descripcion).HasMaxLength(250);
            entity.Property(e => e.Activo).HasDefaultValue(true);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
            entity.Property(d => d.Notas).HasMaxLength(500);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)
                .HasForeignKey<DetalleOrden>(d => d.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<CertificadoCalidad>(entity =>
        {
            entity.ToTable("CertificadosCalidad");
            entity.HasKey(c => c.Id);
            entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
            entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        });
    }
}

```

```

        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
}

}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();

```

```

services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
            "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o =>
o.Certificado).Single(o => o.NumeroOrden == "OF-M2-0001");
global::System.Console.WriteLine($"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} |
Certificado: {cargada.Certificado?.NumeroCertificado}");
}
}

```

## Explicación línea a línea del código acumulativo 2.4

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.4/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad `Observaciones` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad `Planchas` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad `Detalle` de tipo `DetalleOrden?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 21: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad `Certificado` de tipo `CertificadoCalidad?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `}` → cierra el bloque de código actual.

Línea 24: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo del ejercicio.

Línea 25: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 26: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 33: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 34: `}` → cierra el bloque de código actual.

Línea 36: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 37: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 43: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? que EF Core o la lógica de negocio utiliza en este modelo.

Línea 44: `}` → cierra el bloque de código actual.

Línea 46: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 47: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 48: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 49: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 50: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 51: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool que EF Core o la lógica de negocio utiliza en este modelo.

Línea 52: `}` → cierra el bloque de código actual.

Línea 54: `public class DetalleOrden` → declara la clase `DetalleOrden` que forma parte del estado acumulativo del ejercicio.

Línea 55: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 56: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 57: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 58: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad `ComposicionQuimica` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 59: `public double TemperaturaColada { get; set; }` → declara la propiedad `TemperaturaColada` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 60: `public string? Notas { get; set; }` → declara la propiedad `Notas` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 61: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 62: `}` → cierra el bloque de código actual.

Línea 64: `public class CertificadoCalidad` → declara la clase `CertificadoCalidad` que forma parte del estado acumulativo del ejercicio.

Línea 65: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 66: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 67: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 68: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 69: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 70: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 71: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 72: `}` → cierra el bloque de código actual.

Línea 74: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 75: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 76: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 78: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 79: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 80: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 81: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 82: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 83: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 86: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 87: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 88: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 89: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 90: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 91: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 92: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 93: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 94: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 95: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 96: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 97: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 98: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 99: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 100: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 101: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 102: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 103: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 104: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 105: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 106: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 107: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 108: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 109: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 110: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 111: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 112: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 113: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 114: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 115: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 116: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 117: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 118: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 119: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 120: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 121: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 122: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 123: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 124: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 125: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 126: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 127: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 128: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 129: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 130: `entity.HasKey(d => d.Id);` → define la clave primaria de la entidad.

Línea 131: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 132: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 133: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 134: `.WithOne(o => o.Detalle)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 135: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 136: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 137: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 138: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 140: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 141: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 142: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 143: `entity.HasKey(c => c.Id);` → define la clave primaria de la entidad.

Línea 144: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 145: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 146: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 147: `.WithOne(o => o.Certificado)` → establece el otro extremo de referencia de una relación uno-a-uno.

Línea 148: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la propiedad o propiedades de clave foránea.

Línea 149: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 150: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 151: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 153: `}` → cierra el bloque de código actual.

Línea 154: `}` → cierra el bloque de código actual.

Línea 156: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 157: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 158: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 159: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 160: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 161: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 162: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 163: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 164: `}` → cierra el bloque de código actual.

Línea 166: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 167: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 168: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 169: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 171: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 172: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 173: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 174: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 175: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 176: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 177: `}` → cierra el bloque de código actual.

Línea 179: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 180: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 181: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 182: `}` → cierra el bloque de código actual.

Línea 184: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 185: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 186: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 187: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 188: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 189: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 190: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 191: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 192: `}` → cierra el bloque de código actual.

Línea 194: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 195: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 196: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 197: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 198: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 199: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 200: `}` → cierra el bloque de código actual.

Línea 201: `}` → cierra el bloque de código actual.

Línea 203: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 204: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 205: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 206: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 207: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 208: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 209: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 210: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 211: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 213: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 214: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 216: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 217: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 218: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de `AddDbContext`.

Línea 219: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 220: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 221: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 222: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 223: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 224: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 225: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 227: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 228: `services.AddScoped<IServicio0rdenes, Servicio0rdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 230: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 231: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 232: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 233: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 234: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 236: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 237: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 238: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 239: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 241: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 242: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 243: `NumeroOrden = "0F-M2-0001"`, → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 244: `Cliente = "Constructora del Norte"`, → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 245: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 246: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 247: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 248: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 249: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 250: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 251: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 252: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 253: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 254: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 255: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 256: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 257: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 258: `}` → cierra el bloque de código actual.

Línea 259: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 260: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 261: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 262: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 263: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 264: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 265: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 266: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 267: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 268: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 269: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 270: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 271: `}`, → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 272: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 274: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 275: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 276: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 278: `var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o => o.Certificado).Single(o => o.NumeroOrden == "OF-M2-0001");` → incluye de forma eager la navegación relacionada en la consulta.

Línea 279: `global::System.Console.WriteLine($"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} | Certificado: {cargada.Certificado?.NumeroCertificado}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 280: `}` → cierra el bloque de código actual.

Línea 281: `}` → cierra el bloque de código actual.